

# State and Execution: A Precise Definition of the Substrate-Cell Boundary in the Coordination Knowledge Substrate Pattern

**Author:** Wenxin Li (Independent Researcher)

**ORCID:** 0009-0004-8065-3235

**Date:** 25 April 2026

**License:** Creative Commons Attribution 4.0 International (CC BY 4.0)

---

## Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the boundary between the substrate layer and the cell layer as the source paper uses it, so that downstream work can derive from, extend, or argue against the distinction without ambiguity.

## Abstract

The Coordination Knowledge Substrate (CKS) pattern is built from two structural layers — the substrate and the cell — and the source paper states the distinction in §2.1: the substrate is "the atomic unit," the cell is "a coordinated collection of substrates" together with human-authored orchestration rules. The distinction is load-bearing for the rest of the pattern, but the source paper sometimes uses "substrate" loosely to refer to either layer, and the architectural content of several other commitments — conflict preservation, AI-as-substrate-mediator, linear-cost scaling, tool-agnosticism — is only fully readable when the boundary is sharp. This note formalizes the boundary as the source paper commits to it: what each layer is, what each layer commits to, and what crosses the boundary in each direction. It identifies three common conflations that look like CKS but violate the boundary, maps the distinction onto the other commitments, and provides an operational test for whether a system preserves the boundary at all times.

## 1. Why the boundary needs to be named precisely

The CKS pattern develops the substrate and the cell as two distinct objects, but the source paper sometimes uses "substrate" in two senses — sometimes to name the atomic unit (the structured representation that holds coordination content), and sometimes more loosely to name the architectural layer at which CKS sits, including the cells that operate over it. The looser use is harmless in expository passages, but it becomes load-bearing when the pattern's other commitments are taken up. Several CKS commitments are stated in language that implicitly assumes the boundary is sharp, and §5 below shows that the architectural content of each commitment depends on the reader maintaining the distinction. Naming the boundary explicitly — what each layer is, what crosses it, and what cannot — is what makes downstream derivation work possible.

## 2. The substrate layer

In the CKS pattern, the **substrate** is the persistent, structured representation that holds coordination knowledge: entities, relationships, decisions, rationale, and conflicts. It exists across time independently of any particular execution and is the source of truth for *what is the case* about the work the substrate coordinates (§11.3).

The substrate commits to four properties.

**Persistence.** Substrate content exists between cell executions and outlives any one of them; if a piece of state needs to be addressable by future work, it lives in the substrate or it is not addressable.

**Human governance.** Humans retain the rights to inspect, modify, and override any substrate content and any orchestration rule at any time. This is the commitment formalized as a separate derivation in this series; it applies to the substrate layer.

**Determinism and addressability.** Every piece of substrate content carries writer, timestamp, rationale, and provenance. The same substrate state yields the same content; content does not change in response to who is reading it. Path retraceability over the substrate's structure is what makes audit and traceability possible (§3.1).

**Conflict preservation.** Contradictions, inconsistencies, and unresolved ambiguities are first-class substrate state, not anomalies to be silently merged or discarded. The substrate's schema treats conflicts as addressable objects with their own identity and provenance (§5.1).

What the substrate does *not* do is equally part of its definition. The substrate does not execute behavior, apply rules, or make decisions; it holds state, it does not act on state. A substrate that included triggers firing automatically on writes, or rules embedded as side-effects of substrate updates, would be carrying behavior the cell layer is responsible for. A substrate at rest, with no cell executing over it, is just structured content humans can inspect and edit.

### 3. The cell layer

A **cell** is a bounded execution context that operates over substrate content under human-authored orchestration rules. The source paper defines a cell as "a coordinated collection of substrates, possibly just one for a simple task, together with human-authored orchestration rules that determine how the content is used when the cell executes" (§2.1). The cell is the locus where work happens: where the LLM mediator reads from and writes to the substrate, where rules apply, and where outputs are produced.

The cell commits to three properties.

**Rule-governance.** A cell's behavior is determined by orchestration rules — themselves substrate content, authored by humans (§2.3) — rather than by ad-hoc LLM judgment. Rules in CKS apply through cells, not around them.

**Bounded scope.** A cell operates on a defined slice of substrate content for a defined task; both its inputs and its outputs are scoped by the rules that govern its execution. This is the property §6.1 leans on for linear-cost scaling: cell execution cost is proportional to the cell's scope, not to total substrate size.

**Substrate-mediation.** A cell writes its decisions back to the substrate as substrate content, with the cell's identity recorded as the writer. When a cell resolves a conflict under orchestration rules, that resolution is itself recorded as substrate content (§5.3). The cell does not produce ephemeral outputs that change the world without leaving a substrate trace.

What the cell does *not* do is, again, part of its definition. A cell does not hold persistent state across executions; state that needs to persist between cell invocations lives in the substrate, not in the cell. A cell that maintained substrate-relevant state outside the substrate — in the LLM's context window across turns, in cell-local variables that survive between executions, or in any storage humans cannot inspect through the substrate — would produce the failure mode named in §6.2 as *context rot*. The CKS commitment is that such state belongs in the substrate.

#### 4. What crosses the boundary

The boundary between substrate and cell is an interface, and stating what crosses it in each direction is what makes the interface precise.

**From substrate to cell**, three things cross. *Substrate content* — the slice the cell's scope addresses — is read as input. *Orchestration rules* governing the cell's behavior cross with it; rules are themselves substrate content (§2.3), and the cell does not carry a rule set independent of the substrate. *Provenance* crosses with the content: every piece the cell receives is addressable, with writer, timestamp, rationale, and provenance.

**From cell to substrate**, three things cross. *Cell-produced content* is written back as substrate writes, with the cell's identity recorded as writer. *Resolution decisions*, when the cell resolves a conflict under orchestration rules, are recorded as substrate content per the conflict-as-first-class commitment (§5.3); the conflict object itself remains addressable, with the resolution attached. *Updates to existing substrate content* under orchestration-rule authorization cross as substrate writes with the cell's identity.

**What does not cross the boundary** is named explicitly because each is a candidate for a common conflation §6 takes up. *Cell-internal reasoning*, *intermediate state*, and *LLM context* are ephemeral to the cell's execution and do not become substrate content unless explicitly written. *Substrate writes from one cell directly to another cell* do not cross — cells communicate through the substrate, not directly. *Authority* does not cross either; the cell does not gain authority over substrate content by reading it, and the cell's writes are subject to the same human-governance authority all substrate content is. Authority sits at the substrate layer; the cell operates under that authority, not alongside it.

#### 5. How the boundary maps onto the other commitments

The boundary's architectural role is most legible when read across the rest of the pattern's commitments. This section is a brief tour: each commitment has its own dedicated derivation note, and the work here is to show that the boundary is the structural feature each commitment depends on.

*Human-governed maps onto the substrate layer.* The three rights — inspect, modify, override — apply to substrate content and to orchestration rules. Cells do not have an independent governance layer; they operate under rules that are themselves substrate content, and their writes are subject to the same human authority.

*AI-as-substrate-mediator maps onto the boundary itself.* The LLM operates within cells, and its authority is to read from and write to the substrate under orchestration rules (§4.2). The mediator role is what makes the boundary crossable in a controlled way; it is not a third layer.

*Conflict preservation maps onto a split across the boundary.* Contradictions exist at the substrate level (preserved as first-class content); resolution decisions are made at the cell level under rules and recorded back to substrate. Without the boundary, the two-level handling pattern collapses — into substrate-level mutation (which violates preservation) or cell-level state (which violates substrate-as-source-of-truth).

*Linear-cost scaling depends on the boundary.* Cell execution cost is proportional to the cell's scope, not to total substrate size, because cells read selectively. Conflate the two and the cost claim has no scope to attach to.

*Tool-agnosticism's three minimal requirements apply to the substrate's host environment (§7.1), not to the cell's execution environment.* Cells execute through whatever LLM and orchestration mechanism the deployment provides; the substrate's host is what the architecture commits to.

## 6. Common conflations to avoid

Three patterns look like CKS at first reading but violate the substrate-cell boundary. Naming each is what makes the architectural commitment auditable.

**Cell-as-substrate.** Treating a cell's execution context as if it were the substrate. The clearest instance is maintaining "substrate" entirely in the LLM's context across turns — each turn the LLM is shown the prior conversation, treated as the coordination state, and asked to reason over it. This violates persistence (the cell's execution context does not outlive the cell, and what it carried is lost or distorted at the next compaction) and human governance (state living in LLM context is not addressable, inspectable, or override-able outside the cell's execution). The substrate must outlive the cell; cell-as-substrate inverts the relationship.

**Substrate-as-cell.** Treating the substrate as if it had behavior. Examples include embedding rules as triggers in the substrate that fire automatically without an explicit cell execution, attaching side-effects to substrate writes, or giving substrate objects methods that act on other substrate objects when accessed. This violates rule-governance — rules apply through cells, not through substrate magic — and produces a category of failure hard to govern, because the behavior is not associated with a bounded execution context humans can scope and audit.

**Cell-to-cell direct communication.** Cells passing state to each other outside the substrate. Concrete instances include shared in-memory state across cell executions, message-passing channels that carry coordination content between cells without a substrate write, and orchestration mechanisms that hand a cell's output directly to the next cell as input without recording it. This conflation deserves longer treatment than the others because it overlaps with how cells compose into larger workflows — and composition is one of the topics CKS commits to: composition is human-governed and selective at the substrate level, per §6.1, not at the cell level.

The path between cells in CKS runs through the substrate. Cell A writes its output as substrate content, with provenance and rationale; cell B, executing later, reads what cell A wrote. The substrate is the medium. This routing is not incidental — it produces two of the pattern's other commitments at once. It produces traceability, because path retraceability over the substrate's structure (§3.1) is what makes downstream work able to reconstruct what an earlier cell did and why; a cell-to-cell channel outside the substrate is not addressable, and any retraceability over it has to be rebuilt out-of-band. And it produces governance scope: humans inspecting and modifying substrate content can intervene between cell A and cell B because the intervention point is a substrate object, exposed to the same authority that governs the rest of the substrate. Cells passing state outside the substrate forfeit both — the path is no longer addressable, and humans have no inter-cell point at the substrate layer to act on. The system may run; it does not run as CKS.

The composition note in this series treats how cells compose, including the conditions under which composed cell groups exhibit the pattern's properties as a whole. What this note commits to ahead of that fuller treatment is the boundary any such composition must respect: cell-to-cell communication that affects coordination state runs through the substrate, or it is outside the pattern.

## 7. Operational test

A system preserves the substrate-cell boundary if and only if all of the following are true at all times:

1. Substrate content persists across cell executions and is the source of truth for *what is the case* about the work the substrate coordinates.
2. Cells do not hold substrate-relevant state outside the substrate between executions; any state required to persist between cell invocations is written back to the substrate.
3. All cell behavior is governed by orchestration rules that are themselves substrate content, authored by humans and subject to the human-governance commitment.
4. All cell-to-cell communication that affects substrate state runs through the substrate: cells write, other cells read, and the path between them is addressable substrate content.
5. The substrate does not execute behavior; behavior is the cell layer's responsibility, occurring inside bounded, rule-governed cell executions.

A system that fails any of (1)–(5) may be useful and may exhibit one or more CKS-adjacent properties, but does not preserve the substrate-cell boundary in the CKS sense.

## 8. Why naming this boundary matters

The substrate-cell boundary is the structural feature that makes the CKS pattern's other commitments architecturally precise. Each layer commits to categorically different things — the substrate to *being* (persistence, governance, addressability, conflict preservation), the cell to *doing* (rule-governed execution, bounded scope, substrate-mediation) — and the boundary between them is the interface where the LLM, operating inside cells, reads from and writes to the substrate under rules that are themselves substrate content. Naming the boundary explicitly makes the other commitments precisely readable, makes anti-patterns identifiable, and makes downstream derivation work — composition of

cells, hybrid systems that layer CKS against execution substrates and orchestration tools, extensions introducing new cell or substrate content types — defensible from the source paper alone.

Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should use "substrate" and "cell" in the senses formalized here. Subsequent work that uses either term in a different sense is using a different concept, and the difference should be named.

---

### **Source paper**

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

### **How to cite this note**

Li, W. (2026). *State and Execution: A Precise Definition of the Substrate-Cell Boundary in the Coordination Knowledge Substrate Pattern*. 25 April 2026. ORCID: 0009-0004-8065-3235.